



深圳技术大学

SHENZHEN TECHNOLOGY UNIVERSITY

本科毕业论文（设计）

题目：大语言模型驱动的个人知
识库构建与智能优化研究

姓 名：江凯龙

学 院：人工智能学院

专 业：计算机科学与技术

学 号：202200202138

指导教师：池也

职 称：讲师

提交日期：2026 年 5 月 1 日

深圳技术大学本科毕业论文（设计）诚信声明

本人郑重声明：所呈交的毕业论文（设计），题目《大语言模型驱动的个人知识库构建与智能优化研究》是本人在指导教师的指导下，独立进行研究工作所取得的成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式注明。除此之外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。本人完全意识到本声明的法律结果。

毕业论文（设计）作者签名：

日期： 年 月 日

目 录

摘要.....	I
Abstract.....	II
1 绪论	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 检索增强生成	2
1.2.2 个人知识管理	2
1.2.3 知识图谱与智能体记忆	2
1.3 研究内容.....	3
1.4 论文结构.....	4
2 相关技术.....	5
2.1 大语言模型	5
2.2 文本向量化	5
2.3 pgvector.....	6
2.4 RAG 技术.....	7
2.5 LangGraph	8
2.6 Django Ninja 和 Celery.....	9
3 需求分析.....	10
3.1 用户需求.....	10
3.2 功能需求.....	10
3.3 非功能需求	11

4 系统设计	12
4.1 总体架构	12
4.2 模块设计	13
4.3 数据流	14
4.3.1 导入流程	14
4.3.2 查询流程	15
4.4 数据库设计	16
4.4.1 Document 表	16
4.4.2 DocumentSummary 表	17
4.4.3 DocumentChunk 表	17
4.4.4 ChunkEmbedding 表	17
4.4.5 Memory 表	18
4.4.6 WikiPage 表	18
4.4.7 SourceSummary、WikiIndex 与 WikiLog 表	19
4.5 检索策略	19
4.6 Prompt 设计	20
4.6.1 智能体系统提示词	20
4.6.2 Memory 更新提示词	21
4.6.3 Wiki 构建提示词	22
4.6.4 文档摘要提示词	23
4.7 智能体设计	23

5 系统实现.....	25
5.1 开发环境.....	25
5.2 文档导入.....	25
5.3 文本分块.....	26
5.4 向量化.....	27
5.5 Memory 更新.....	28
5.6 Wiki 构建.....	28
5.7 检索实现.....	29
5.8 智能体问答.....	29
5.9 文档摘要.....	30
5.10 前端实现.....	30
6 测试与评估.....	32
6.1 功能测试.....	32
6.2 检索效果.....	32
6.3 生成质量.....	36
7 总结与展望.....	38
7.1 工作总结.....	38
7.2 不足.....	38
7.3 展望.....	39
参考文献.....	40
致谢.....	41

大语言模型驱动的个人知识库构建与智能优化研究

【摘要】 在个人博客，技术笔记等数字化个人文档资料管理的场景下，用户会积累越来越多的文档资料。但这些高价值的文本信息一直缺乏一种能够适用于长期积累与较大规模数据场景下的有效管理和汇聚方法。随着记录的博客和笔记数量越来越多，这些文档资料往往会变得越来越零散，甚至会被用户遗忘或丢弃掉，因为对于用户来说，随着博客和笔记数量的不断增加，人工手动整理和维护的成本也不断增加。

为了解决大量的个人非结构化文本资料的管理难题，本文设计并实现了一个由大语言模型驱动的个人知识库构建与优化系统 MindTrace。能够把原始文档信息，即用户的博客，笔记或任何其它文本资料，通过大语言模型处理流水线把用户原始文档资料沉淀为一个个实体（Entity），观念（Concept），以及主题（Topic）。并且依托异步任务解耦技术，让用户无感于后台基于 LLM 的文档数据处理流水线。

结果表明，MindTrace 在个人文档管理场景下是行之有效的，系统根据文档中的观念和主题自行智能归类，免去人工自行管理的烦恼，并从看似零散的个人文档资料中提取出有价值的信息，进一步识别文档与文档之间的观念，主题联系，得到用户长期形成的观念脉络，为用户长期知识沉淀提供一种解决办法。

【关键词】 大语言模型；个人知识库；智能优化；知识图谱

Research on LLM-driven Personal Knowledge Base Construction and Intelligent Optimization

【Abstract】 In scenarios involving the management of digital personal documents such as personal blogs and technical notes, users accumulate increasingly larger amounts of documentation. However, this high-value textual information has consistently lacked an effective management and aggregation method suitable for long-term accumulation and large-scale data scenarios. As the number of blog posts and notes grows, these documents often become increasingly fragmented, and may even be forgotten or discarded by users. This is because, for users, the cost of manual organization and maintenance increases with the number of blog posts and notes.

To address the challenge of managing large amounts of unstructured personal textual data, this paper designs and implements MindTrace, a personal knowledge base construction and optimization system driven by a large language model. It can transform raw document information—users’ blogs, notes, or any other textual data—into entities, concepts, and topics through a large language model processing pipeline. Furthermore, relying on asynchronous task decoupling technology, it allows users to remain unaware of the background LLM-based document data processing pipeline.

The results show that MindTrace is effective in personal document management scenarios. The system can intelligently classify documents based on their concepts and themes, eliminating the hassle of manual management. It can also extract valuable information from seemingly scattered personal documents, further identify the connections between concepts and themes between documents, and obtain the conceptual framework formed by users over a long period of time, providing a solution for users’ long-term knowledge accumulation.

【Key words】 Large Language Model; personal knowledge base; intelligent optimization; knowledge graph

1 绪论

1.1 研究背景与意义

在现代社会，个人脑力工作者每天都在产生大量的文本内容——博客文章、技术笔记、项目文档、生活分享。这些内容是个人经验和智慧的结晶，具有极高的价值。然而，现实情况是：这些宝贵的知识往往“写完即丢”，难以被有效的组织和复用。

近年来，大语言模型（Large Language Model, LLM）的快速发展为个人知识管理带来了新的可能性^[1-2]。LLM 不仅能够理解自然语言，还能进行信息抽取、知识归纳和智能问答。结合检索增强生成（Retrieval-Augmented Generation, RAG）技术，系统可以在回答问题前先检索相关文档，再基于检索结果生成回答，从而大幅降低模型“幻觉”的风险。然而，现有的 RAG 系统大多将文档视为同质化的知识源，缺乏对知识的结构化组织和长期记忆能力。

从技术演进的角度看，个人知识管理工具经历了三个发展阶段：第一阶段是文件系统时代，用户通过文件夹和文件名组织知识，检索依赖精确的路径记忆；第二阶段是笔记软件时代，Notion、OneNote 等工具提供了标签和全文搜索功能，但仍停留在字面匹配层面；第三阶段是智能知识库时代，借助 LLM 和 RAG 技术，系统能够理解语义、发现关联、提供个性化回答。本文的工作正处于第三阶段的探索前沿。

针对以上问题，本文提出 MindTrace 项目，一个大语言模型驱动的个人知识库构建与智能优化系统。MindTrace 的核心创新在于：(1) 实现了基于 LLM 的文本资料处理流水线，免去用户自行维护大量文档资料的烦恼。(2) 基于文本资料处理流水线的结果，得出文档之间的关系，也就是哪些文档说的是一件事，或说的不是一件事但从更高角度看是作者的一个想法产生的，帮助用户发现文档之间的潜在关系和思想脉络。(3) 通过提示词设计，算法设计和工具编排，构建在长期层面也稳定有效的基于 LLM 的文本资料处理流水线，为用户长期积累和沉淀文档资料奠定扎实的基础。

从应用价值来看，MindTrace 系统的意义体现在三个层面：在个人层面，系统帮助知识工作者构建“第二大脑”，实现知识的高效检索和复用；在技术层面，系

统验证了 RAG、知识图谱和智能体技术在个人知识管理场景下的可行性；在学术层面，系统为 LLM 驱动的知识管理系统提供了设计参考和工程实践案例。

1.2 国内外研究现状

1.2.1 检索增强生成

检索增强生成（Retrieval-Augmented Generation, RAG）是近年来大语言模型应用领域的重要突破。Lewis 等人^[3]首次提出 RAG 的概念，通过“先检索再生成”的范式，一定程度上解决了 LLM 无法获取外部知识和幻觉问题。此后，研究者们从多个维度对 RAG 进行了改进：Self-RAG^[4]引入了模型自反思机制，使系统能够动态判断检索时机和结果质量；DPR^[5]等稠密检索方法显著提升了语义匹配的精度；结合知识图谱的研究^[6]则探索了结构化知识对检索质量的增强作用。

然而，现有 RAG 系统普遍存在一个局限：缺乏对文档与文档之间的逻辑关系的推理，技术黑盒（人脑无法解释结果），以及会污染用户的上下文，塞入过长的低价值信息。本文提出的 MindTrace 系统通过引入结构化 Wiki 层和用户记忆机制，有效弥补了这一不足。

1.2.2 个人知识管理

个人知识管理（Personal Knowledge Management, PKM）是一个历史悠久的研究方向。从早期的 Evernote、OneNote，到近年的 Notion、Obsidian，工具在不断演进，但核心问题始终是：如何让散落的知识变得可检索、可关联、可复用。孔德超^[7]较早地系统论述了个人知识管理的内涵与方法，为后续研究奠定了理论基础。

大语言模型的出现为 PKM 带来了新的机遇。相关智能体研究表明，语言模型能够在推理过程中自主调用外部工具^[8-9]，这为构建智能化的知识助手奠定了基础。MindTrace 正是在这一趋势下的探索性成果。

1.2.3 知识图谱与智能体记忆

知识图谱通过实体、关系和属性组织非结构化知识，是支撑语义查询和知识推理的重要手段^[10]。近年来，利用 LLM 自动构建知识图谱成为研究热点^[11-12]，这

为本文的 Wiki 构建提供了方法论参考。

在智能体记忆方面，现有系统多依赖扁平的向量索引，难以表达多跳关联和结构约束。ReAct^[8] 和 Toolformer^[9] 等工作展示了工具调用式智能体的潜力。本文在此基础上，设计了结构化 Memory 机制，将用户的工作背景、近期关注和长期信息整合为智能体的上下文，实现了真正意义上的”长期记忆”。

总体来看，RAG、个人知识管理、知识图谱和智能体记忆已形成多个成熟的研究方向。然而，将这些技术有机整合，构建一个能够持续演化、具备结构化知识组织能力的个人知识库系统，仍是一个开放性問題。本文的工作正是对这一问题的积极探索和工程实践。

1.3 研究内容

本文围绕个人知识库的构建与智能问答展开，主要完成了以下工作：

(1) 设计并实现了多层次知识表示体系。系统将知识组织为四个层次：原始文档层、向量检索层、结构化 Wiki 层和用户记忆层。这种分层设计保留了用户的原始文档，以便后续结果归因，打破 RAG 技术的黑盒。同时后面几层都是基于原始文档层的高级抽象表现，最大限度的提高文本的信息密度。

(2) 提出智能化的知识演化机制。系统能够自动从导入的文档中提取实体、概念和主题，构建结构化的 Wiki 知识页面。Wiki 层支持实体、概念、主题三级分类，并具备自动合并、重命名和重组能力，实现了知识的持续沉淀和演化。

(3) 设计了基于 LangGraph 的智能体架构。智能体能够自主选择 and 编排 Wiki 检索、向量检索、文档查询、图片生成等多种工具，将工具返回的结果作为上下文生成回答。这种架构使系统具备了处理复杂查询的能力。

(4) 实现了结构化 Memory 机制。系统维护用户记忆，包含工作背景、个人背景、近期关注 and 历史信息。Memory 在每次文档导入后自动更新，并在智能体回答时注入系统提示词，实现了真正意义上的”长期记忆”。

(5) 构建了完整的工程化系统。后端采用 Django Ninja 框架，使用 PostgreSQL + pgvector^{PKG} 存储数据，通过 Celery 实现异步任务调度；前端采用 SvelteKit 框架，提供了文档管理、智能对话、Wiki 浏览、Memory 查看等功能。

1.4 论文结构

本文共七章。

第一章绪论，介绍背景和研究内容。

第二章相关技术，介绍本文用到的技术栈。

第三章需求分析，梳理系统的功能和非功能需求。

第四章系统设计，描述架构、数据库和核心流程的设计方案。

第五章系统实现，写各模块的具体实现细节。

第六章测试与评估，对系统进行功能验证和效果评估。

第七章总结与展望，总结工作并讨论不足和改进方向。

2 相关技术

2.1 大语言模型

Transformer 架构^[1] 的提出，彻底改变了自然语言处理的范式。大语言模型（Large Language Model, LLM）正是建立在这一架构之上的大规模预训练语言模型，通过在海量文本数据上进行训练，模型习得了丰富的语言知识和世界知识。从 GPT 系列到 LLaMA、Qwen 等开源模型，LLM 的能力边界不断拓展，在文本理解、生成、推理和代码编写等方面展现出惊人的能力。

对于本文的个人知识库系统而言，LLM 的三个核心能力至关重要：

（1）文本理解能力：模型能够从非结构化文本中提取关键信息、识别实体关系、归纳主题概念。这一能力是系统实现文档摘要、实体抽取和知识归类的基础。例如，给定一段技术文档，模型能够识别出其中涉及的技术栈、项目名称、解决方案等关键实体。

（2）文本生成能力：模型能够根据指令和上下文生成连贯、准确的文本。这一能力使系统能够基于检索到的历史文档生成个性化的回答，实现真正意义上的智能问答。生成过程需要平衡信息的准确性和表达的自然性，这对模型的能力提出了较高要求。

（3）指令遵循能力：模型能够理解并执行结构化指令，输出符合预定义格式的结果。这一能力对于 Memory 更新和 Wiki 构建至关重要——系统需要模型输出规范的 JSON 结构或分类结果，而不是自由文本。

无特殊说明默认使用阿里的 qwen3.6-plus 模型，通过 DashScope 平台的 OpenAI 兼容 API 调用。该模型在中文文本理解和生成方面表现出色，且在国内网络环境下具有良好的可用性和性价比。选择该模型的另一个重要原因是其与 tongyi-embedding-vision-plus 向量模型同属于阿里生态，调用便捷。

2.2 文本向量化

文本向量化（Text Embedding）是将自然语言文本转换为向量数学表示的技术。通过 embedding，语义相近的文本在向量空间中距离较近，语义不同的文本距

离较远，从而实现了文本的“语义指纹”表示。这一技术是语义检索的基础，也是将非结构化文本转化为机器可理解形式的关键步骤。

从技术原理上看，文本向量化模型通常采用对比学习的方式训练：给定一个句子和相关的正样本对，模型学习将正样本映射到相近的向量位置，将负样本推远。通过在大规模文本语料上训练，模型习得了丰富的语义理解能力，能够捕捉文本的深层语义信息。

本文选用通义千问的 `tongyi-embedding-vision-plus` 模型，输出维度为 1152。该模型支持多模态输入（文本、图片、视频），但本文仅使用文本向量化功能。选择该模型的原因在于：(1) 与 `qwen3.6-plus` 同属阿里生态，调用便捷；(2) 中文文本向量化效果优异，能够准确捕捉中文语义；(3) 1152 维的向量维度在表达能力和计算效率之间取得了良好平衡，既保证了足够的语义表达能力，又不会因维度过高而导致计算和存储开销过大。

向量相似度的计算采用余弦相似度（Cosine Similarity），其定义见公式 (4.1)。该方法通过计算两个向量的夹角余弦值来衡量相似性，取值范围为 $[-1, 1]$ ，值越大表示越相似。余弦相似度计算简单、效果稳定，是向量检索中最常用的相似度度量方法。与其他相似度度量方法（如欧氏距离、曼哈顿距离）相比，余弦相似度对向量的绝对长度不敏感，更适合处理文本向量的相似性计算。

2.3 pgvector

`pgvectorPKG` 是 PostgreSQL 的向量扩展，使传统的 RDB 也可以高性能的支持向量存储与检索。与独立的向量数据库（如 Qdrant、Milvus）相比，`pgvectorPKG` 的优势在于：(1) 无需额外部署和维护向量数据库，降低了系统复杂度；(2) 可以与关系型数据在同一事务中操作，保证数据一致性；(3) 支持 SQL 查询，便于与其他数据关联。

从技术实现角度看，`pgvectorPKG` 通过在 PostgreSQL 中添加新的数据类型 `Vector`，支持存储和查询高维向量数据。在查询时，`pgvectorPKG` 提供了余弦距离、欧氏距离和内积三种相似度度量方式，用户可以根据业务需求选择合适的度量方法。在索引方面，`pgvectorPKG` 支持 HNSW 和 IVFFlat 两种索引类型。HNSW 索引基于分层可导航小世界图算法，在查询精度和速度之间取得了良好平衡；IVFFlat 索引基于倒排文件索引，适合大规模数据的近似最近邻搜索。

本文选择 `pgvector`^{PKG} 而非独立向量数据库，是基于工程实践的务实考量。MindTrace 系统除了向量数据外，还需要存储文档、摘要、Memory 和 Wiki 页面等结构化数据。使用单一 PostgreSQL 实例管理所有数据，可以简化部署、备份和同步流程，降低运维成本。此外，`pgvector`^{PKG} 的 SQL 兼容性使得向量检索可以与关系查询无缝结合，例如在检索时同时过滤文档来源、时间范围等条件，这在独立向量数据库中实现起来相对复杂。

本文的数据规模（几千条记录）下，全量余弦距离排序的性能完全满足需求。

2.4 RAG 技术

检索增强生成（Retrieval-Augmented Generation, RAG）是一种将检索与生成相结合的技术范式。其核心流程为：用户提问 → 问题向量化 → 向量检索 → 拼接 prompt → LLM 生成回答。这一范式有效解决了 LLM 知识时效性和幻觉问题，使模型能够基于外部知识生成更准确、更有依据的回答。

RAG 技术的出现具有里程碑意义。在 RAG 之前，LLM 的知识完全来自训练数据，存在两个根本性问题：一是知识时效性，模型无法获取训练截止日期之后的新知识；二是幻觉问题，当遇到训练数据未覆盖的问题时，模型可能编造看似合理但实际错误的答案。RAG 通过“先检索再生成”的机制，将模型的回答建立在真实文档的基础上，从根本上解决了这两个问题。

在个人知识库场景下，RAG 的价值尤为突出。传统 LLM 只能给出通用回答，而 RAG 使系统能够引用用户自己写过的内容，提供基于个人历史经验的个性化回答。这是从“通用回答”到“个人回答”的质变。例如，当用户询问“我之前怎么解决部署问题”时，系统能够检索到用户过去写的部署相关文档，基于这些真实经历给出回答，而不是泛泛而谈部署的最佳实践。

然而，标准 RAG 存在明显局限：(1) 无法知悉文档与文档之间的内在联系 (2) RAG 结果黑盒，人脑无法理解最后的结果为什么是这样 (3) 缺乏对用户背景的长期建模能力；(4) 检索策略单一，仅依赖语义相似度，未考虑时间、个人背景等因素。针对这些局限，本文通过引入结构化 Wiki 层、用户记忆机制和 LangGraph 智能体进行了系统性改进。改进效果将在后续测试与评估章节进行验证。

2.5 LangGraph

LangGraph 是 LangChain 生态中用于构建智能体的框架,它通过状态图(State-Graph)定义智能体的工作流程。与传统 RAG 的固定流程(检索 → 拼接 → 生成)不同,LangGraph 允许 LLM 自主决定调用哪些工具、按什么顺序调用,实现了从”被动响应”到”主动推理”的转变。

从设计理念上看,LangGraph 体现了”LLM 即控制器”的思想:LLM 不仅负责生成回答,还负责决策和编排。这种设计使系统具备了处理复杂查询的能力——当一个问题需要多步检索、多源信息整合时,智能体能够自主规划检索策略,而不是依赖预设的固定流程。

LangGraph 的核心概念包括:

(1) 状态图 (StateGraph): 定义了节点和边,节点是处理步骤,边是转移条件。本文的智能体包含两个节点: `call_model` (调用 LLM) 和 `tools` (执行工具调用),结构简洁但功能完备。状态图的设计遵循了”最小节点”原则——只包含必要的处理步骤,避免过度复杂的图结构。

(2) 工具 (Tool): 智能体可以调用的外部功能。本文定义了八个工具,具体如表 4-7 所示,覆盖了知识库查询的主要场景:获取 Wiki 索引、搜索 Wiki 页面、读取 Wiki 页面、向量检索、获取分块内容、获取文档内容、获取文档摘要和图片生成。工具的设计遵循了”单一职责”原则——每个工具只负责一个特定的功能,便于维护和扩展。

(3) 检查点 (Checkpoint): 保存对话状态,支持多轮交互。本文使用 `InMemorySaver` 实现内存级状态持久化。检查点机制使智能体能够在多轮对话中保持上下文连续性,不会因为单次查询结束而丢失对话历史。

智能体的工作流程为:接收用户消息 → LLM 判断是否需要调用工具 → 如果需要,执行工具调用并将结果返回给 LLM → LLM 继续判断,直到不需要调用工具 → 输出最终回答。这一循环机制使智能体能够处理需要多步检索的复杂问题。例如,当用户询问”我对 RAG 技术的理解”时,智能体可能先调用 Wiki 工具获取 RAG 相关的知识页面,再调用向量检索工具获取更多细节,最后综合这些信息生成回答。

2.6 Django Ninja 和 Celery

Django Ninja 是 Django 框架的 Web API 库，用于构建 RESTful API。相比 Django REST Framework，Django Ninja 的优势在于：(1) 使用 Python 类型注解定义请求和响应结构，代码更简洁，类型安全性更高；(2) 自动生成 OpenAPI 文档，便于前后端协作，减少了文档维护成本；(3) 基于 Pydantic 进行数据验证，错误提示更加友好；(4) 开发效率更高，适合快速迭代的项目。

从技术架构角度看，Django Ninja 采用了“约定优于配置”的设计哲学，通过装饰器和类型注解定义 API 端点，减少了样板代码。这种设计使开发者能够将更多精力集中在业务逻辑上，而不是框架配置上。对于 MindTrace 这样的个人项目，开发效率是技术选型的重要考量因素。

Celery 是 Python 的异步任务队列框架，用于处理耗时操作。在 MindTrace 系统中，文档导入后的分块、向量化、摘要生成、Memory 更新和 Wiki 构建等操作均通过 Celery 异步执行。这种设计的优势在于：(1) 用户上传文档后可以立即得到响应，无需等待后台处理完成，提升了用户体验；(2) 后台任务可以在独立的 Worker 进程中执行，不影响主服务的响应性能；(3) 支持任务重试和错误处理，提高了系统的可靠性；(4) 可以根据负载情况动态调整 Worker 数量，实现水平扩展。

从系统架构角度看，Celery 的引入实现了“请求-处理”的解耦：API 层只负责接收请求和创建任务，具体的处理逻辑由 Worker 异步执行。这种架构使系统能够更好地应对突发流量，避免因为大量文档导入请求导致 API 层响应变慢。

Celery 使用 Redis 作为消息代理（Broker），Redis 的 ACK 机制为消息队列的任务重试，确认提供原生支持，简化了系统架构。Redis 的高性能和低延迟特性，保证了任务调度的实时性，适合对响应速度要求较高的场景。

3 需求分析

3.1 用户需求

本文的用户画像定位于平时会产出文档脑力工作者，也可以是会用键盘记录文字的任何人。随着知识积累，传统管理方式的弊端日益凸显，具体需求如下：

（1）语义化检索需求。传统的文件夹分类和关键词搜索只能进行字面匹配，无法理解语义层面的关联。用户需要一种能够理解自然语言意图的检索方式，例如“我之前怎么解决部署问题”能够匹配到包含“上线”、“发布”等相关内容的文档。这种需求在实际工作中非常普遍——技术人员往往记不清原文的精确用词，但记得大概的意思和场景。

（2）自动化知识整理需求。手动为文档打标签、做分类是一项繁琐且难以持续的工作。用户需要系统能够自动从文档中提取关键信息、识别实体概念、归纳主题分类，实现知识的自动化组织。这种需求的根源在于：知识积累是一个持续的过程，随着文档数量的增加，手动整理的成本会越来越高，最终变得不可持续。

（3）个性化问答需求。通用 LLM 只能提供通用回答，无法基于用户的个人历史经验给出针对性建议。用户需要系统能够引用自己过去写过的内容，提供“基于我自己经历的回答”。例如，当用户询问“我对 RAG 技术的理解”时，系统应该能够找到用户过去写的 RAG 相关文章，基于这些文章给出回答，而不是泛泛介绍 RAG 的概念。

3.2 功能需求

基于用户需求分析，系统功能需求可归纳为以下五个需求：

（1）原始文档数据导入，支持对原始文本数据的 RESTful API 操作，所有会触发 LLM 流水线或其它耗时任务的操作都会通过 `celery` 异步执行，保证用户的使用体验和系统的稳定性。

（2）原始文档处理，基于 LLM 处理流水线提取出 Entity（实体），Concept（观念），Topic（主题）。文档上传后自动触发异步处理流水线，包括文本分块、向量化、摘要生成、Memory 更新和 Wiki 组件构建。这些操作在后台 Celery Worker 异

步执行，用户无需等待。

(3) 传统 RAG 需求，部分 agent 对话场景下，agent 可能想要获取关于某个问题相近的原始材料来做更精确的分析，故保留传统的 RAG 功能，即基于 query 的 embedding 与用户文档分块的 embedding 做相似度检索，返回相似度最高的前 N 个分块内容。

(4) 基于用户文档资料的个性化的 agent 对话体验，用户可以通过智能对话界面提出问题，智能体能够基于用户的文档资料进行检索和推理，给出个性化的回答。智能体能够自主选择调用 Wiki 检索、向量检索、文档查询等工具，将工具返回的结果作为上下文生成回答。

(5) 前端 Web 展示界面，提供友好便捷的操作界面。基于 SvelteKit 构建的现代化前端界面，提供文档管理、智能对话、Wiki 浏览、Memory 查看等功能。

3.3 非功能需求

(1) 可扩展性：系统架构遵循开闭原则，对扩展开放、对修改关闭。后续可能需要添加新的工具、支持新的文档格式，架构设计需要预留扩展点。

(2) 可解释性：回答应该能够追溯到具体的文档来源，用户可以验证系统回答的依据。这是建立用户信任的基础——系统说“你之前写过...”，用户需要能看到原文。

(3) 数据隐私：系统部署在用户自己的服务器上，数据不上传到第三方平台。Embedding 和 LLM 的 API 调用是当前技术条件下的必要妥协，后续可通过部署本地模型实现完全私有化。

(4) 响应速度：文档处理采用异步机制，问答采用流式输出，最大限度减少用户等待感。良好的用户体验是系统被持续使用的前提。

4 系统设计

4.1 总体架构

系统采用前后端分离的分层架构，如图 4-1 所示。

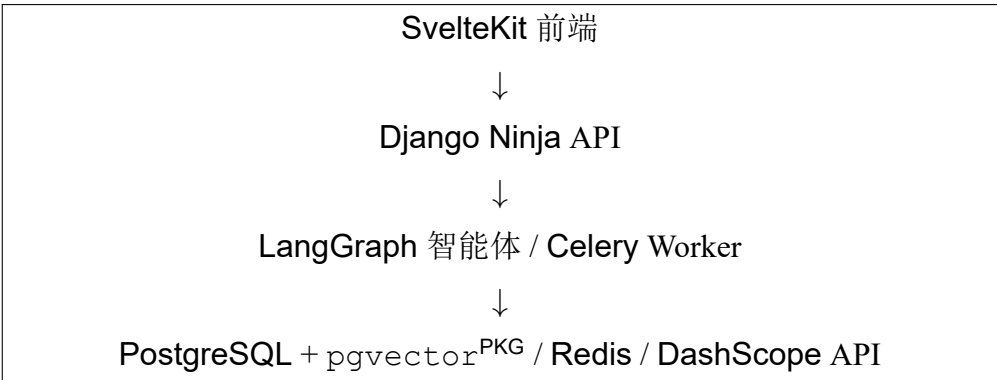


图 4-1 系统总体架构

架构自上而下分为四层：

（1）展示层：基于 **SvelteKit** 构建的前端应用，负责页面渲染和用户交互。**SvelteKit** 的轻量级特性和优秀的编译优化，保证了良好的用户体验。展示层通过 **RESTful API** 与服务层通信，实现了前后端的解耦。**SvelteKit** 的响应式设计使得页面能够实时更新，为用户提供了流畅的交互体验。

（2）服务层：**Django Ninja API** 层，处理前端请求，提供文档管理、检索查询等 **RESTful** 接口。服务层是系统的入口，负责请求的接收、验证和路由。通过 **Django Ninja** 的类型注解和自动文档生成能力，服务层提供了清晰的 **API** 接口规范，便于前后端协作开发。

（3）智能层：包括 **LangGraph** 智能体和 **Celery Worker**。智能体负责编排工具调用和生成回答，实现了从“被动响应”到“主动推理”的转变；**Worker** 负责执行文档处理的异步任务，包括分块、向量化、摘要生成、**Memory** 更新和 **Wiki** 构建。智能层的设计体现了“异步优先”的原则——将耗时操作从请求-响应周期中剥离，保证了系统的响应性能。

（4）数据层：**PostgreSQL + pgvector^{PKG}** 存储所有结构化数据和向量数据，**Redis** 作为消息代理和缓存层，**DashScope API** 提供 **LLM** 和 **Embedding** 服务。数据层的设计遵循了“单一数据源”原则——所有数据存储在 **PostgreSQL** 中，避免

了数据同步的复杂性。pgvector^{PKG} 的引入使得向量数据和结构化数据可以在同一事务中操作，保证了数据一致性。

这种分层架构的优势在于：(1) 各层职责清晰，便于开发和维护；(2) 前后端分离，支持独立部署和扩展；(3) 异步任务处理，保证了系统的响应性能；(4) 单一数据源，简化了数据管理；(5) 模块化设计，便于功能扩展和技术栈升级。

4.2 模块设计

系统划分为以下八个功能模块：

(1) 文档管理模块：实现 Document 模型，即用户原始文档，的管理操作，这是系统的数据基础。后续所有的处理和 Wiki 持续集成构建都是基于 Document 进行的，因此 Document 是不可修改的，便于后续 Wiki 构建结果归因和追溯。

(2) 文档处理模块：文档 Document 导入后的自动化处理流水线，包括文本分块、向量化、摘要生成、Memory 更新和 Wiki 持续构建五个步骤。相关流程以 Celery 任务的形式异步执行，是系统最复杂的模块。

(3) 向量检索模块：基于 pgvector^{PKG} 的余弦相似度检索，检索范围为 Document 分块的结果，分块可以更精确的检索到与 query 相关的那一部分内容。同时作为智能体的工具和独立的 API 端点暴露，实现了”双重身份”设计。

(4) Wiki 知识模块：将文档 Document 整理为 SourceSummary、WikiPage、Wiki-Index 和 WikiLog，支持实体 (Entity)、概念 (Concept)、主题 (Topic) 三个层级的知识页面管理。这是系统的核心创新点——将非结构化文档转化为结构化知识。

(5) Memory 模块：维护单例 Memory 记录，用 JSON 结构保存用户工作背景、个人背景、近期关注和历史信息，并在智能体系统提示词中注入，实现了”长期记忆”能力。

(6) 智能体模块：基于 LangGraph 的 StateGraph，绑定表 4-7 所示的八个工具（文档检索、Wiki 检索、文档读取、图片生成等），支持多轮对话和工具调用。

(7) 内容生成模块：支持文档摘要、Wiki 页面内容和 Slidev 演示文稿的自动生成。

(8) 前端模块: SvelteKit 应用, 包含文档管理、智能对话、Wiki 浏览、Memory 查看、文档上传等页面。

4.3 数据流

4.3.1 导入流程

用户上传文档后的处理流程如图 4-2 所示。

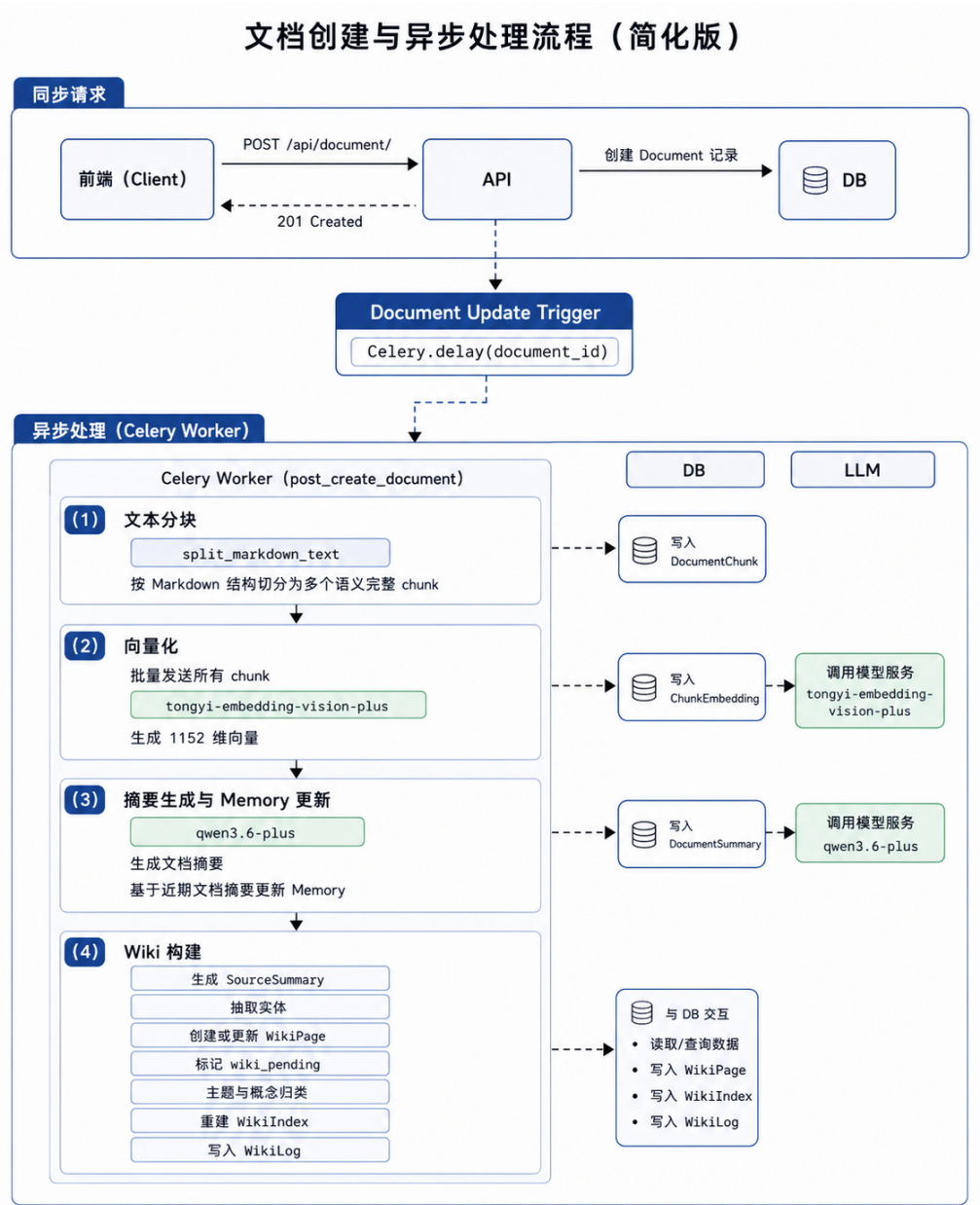


图 4-2 文档导入流程

前端调用 POST /api/document/ 接口，接口创建 Document 记录后通过

`Celery.delay()` 触发 `post_create_document` 异步任务。该任务依次执行以下四类工作：

（1）文本分块：调用 `split_markdown_text` 函数，基于 Markdown 结构将文档切分为多个语义完整的 `chunk`，每个 `chunk` 创建一条 `DocumentChunk` 记录。

（2）向量化：将所有 `chunk` 批量发送至 `tongyi-embedding-vision-plus` 模型，获取 1152 维向量，存入 `ChunkEmbedding` 表。

（3）摘要生成与 Memory 更新：调用 `qwen3.6-plus` 模型生成文档摘要，存入 `DocumentSummary` 表，并基于近期文档摘要更新 Memory。

（4）Wiki 构建：生成 `SourceSummary`，抽取实体并创建或更新 `WikiPage`，将文档标记为 `wiki_pending` 后进行主题和概念归类，最后重建 `WikiIndex` 并写入 `WikiLog`。

前两步和摘要生成在 `post_create_document` 任务中串行执行，Wiki 构建通过单独的 Celery 任务触发。这种设计既保证了处理的完整性，又实现了任务的解耦。

4.3.2 查询流程

用户提问后的处理流程如图 4-3 所示。



图 4-3 查询流程

前端调用 `POST /api/agent/chat/stream` 接口，接口将用户消息传递给 `LangGraph` 智能体。智能体的系统提示词中注入了 `Memory` 模块格式化后的用户背景信息，包括工作背景、个人背景、近期关注和历史信息。

智能体根据问题的性质自主决定是否调用工具。对于涉及结构化知识的问题，优先调用 `get_wiki_index`、`search_wiki_pages` 和 `get_wiki_page` 等 Wiki 工具；当 Wiki 页面不足以回答问题时，则调用 `query_document_chunks` 进行文档级向量检索。工具返回的结果会追加到消息列表中，智能体基于这些结果生成最终回答。回答通过 SSE（Server-Sent Events）流式返回给前端，实现了实时的交互体验。

4.4 数据库设计

数据库设计是系统架构的核心环节，直接影响系统的性能、可扩展性和数据一致性。MindTrace 系统的数据库设计遵循以下原则：

（1）范式化与反范式化的平衡：在保证数据一致性的前提下，适当采用反范式化设计提升查询性能。例如，DocumentChunk 表中冗余存储了 chunk_index 字段，避免了在查询时需要额外的排序操作。

（2）外键约束与级联删除：通过 Django 的 on_delete 参数定义外键约束，保证数据的引用完整性。例如，删除 Document 时，关联的 DocumentSummary、DocumentChunk 和 ChunkEmbedding 会被自动级联删除，避免了孤立数据的产生。

（3）向量字段与关系字段的分离：将向量数据（ChunkEmbedding）与文本数据（DocumentChunk）分离存储，既保证了向量检索的性能，又便于独立管理向量数据。

系统有多张核心表，但都是围绕着 Document 用户原始文档表来建立的，可以分为文档检索、Memory 和 Wiki 三类。下面分别说明。

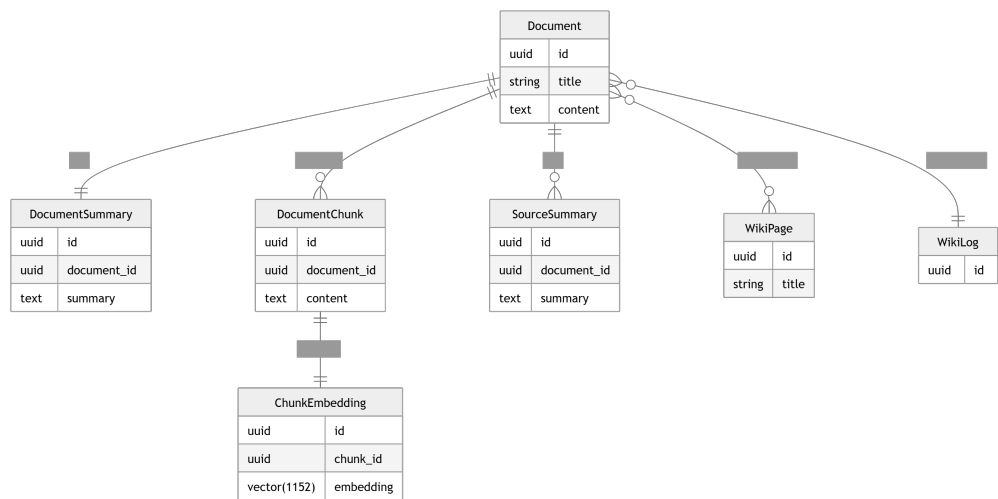


图 4-4 数据库设计

4.4.1 Document 表

存储用户上传的原始文档，结构如表 4-1 所示。

表 4-1 Document 表

字段	类型	约束	说明
id	BigAutoField	主键	自增 ID
uuid	UUIDField	唯一索引	文档标识，用于 API 交互
title	CharField(255)		文档标题
content	TextField		文档全文
source	CharField(255)	默认 'unknown'	来源
wiki_pending	BooleanField	默认 false	是否等待 Wiki 归类
created_at	DateTimeField	auto_now_add	创建时间
updated_at	DateTimeField	auto_now	更新时间

uuid 字段是额外加的，用于 API 中标识文档。用 UUID 而不是自增 ID，主要是考虑到后面如果有数据迁移或合并的场景，UUID 不会冲突。

4.4.2 DocumentSummary 表

存储文档的自动摘要，跟 Document 一对一关联，结构如表 4-2 所示。

表 4-2 DocumentSummary 表

字段	类型	约束	说明
id	BigAutoField	主键	自增 ID
document	OneToOneField	关联 Document	所属文档
content	TextField		摘要文本
created_at	DateTimeField	auto_now_add	创建时间
updated_at	DateTimeField	auto_now_add	更新时间

摘要限制在 30 个词以内。这个长度足够概括一篇博客的核心内容，又不会太长。

4.4.3 DocumentChunk 表

存储文档的分块，跟 Document 多对一关联，结构如表 4-3 所示。

4.4.4 ChunkEmbedding 表

存储分块的向量，跟 DocumentChunk 一对一关联，结构如表 4-4 所示。

向量维度 1152 是 tongyi-embedding-vision-plus 模型的固定输出，不能改。

表 4-3 DocumentChunk 表

字段	类型	约束	说明
id	BigAutoField	主键	自增 ID
document	ForeignKey	关联 Document	所属文档
content	TextField		分块文本
chunk_index	IntegerField		分块序号，从 0 开始
created_at	DateTimeField	auto_now_add	创建时间

表 4-4 ChunkEmbedding 表

字段	类型	约束	说明
id	BigAutoField	主键	自增 ID
document_chunk	OneToOneField	关联 DocumentChunk	所属分块
embedding	VectorField(1152)		向量
embedding_model	CharField(64)	默认'tongyi-embedding-vision-plus'	模型名
created_at	DateTimeField	auto_now_add	创建时间

4.4.5 Memory 表

Memory 表存储系统维护的用户长期记忆，当前实现采用单例模式，主键固定为 1。其结构如表 4-5 所示。

表 4-5 Memory 表

字段	类型	约束	说明
id	BigAutoField	主键	单例记录 ID
data	JSONField	默认结构	用户长期记忆 JSON
updated_at	DateTimeField	auto_now	更新时间

data 字段包含 user 和 history 两个主要部分。user 下包括 workContext、personalContext 和 topOfMind，用于保存工作背景、个人背景和近期关注；history 下包括 recentMonths、earlierContext 和 longTermBackground，用于保存近期动态、历史回顾和长期背景。系统通过 LLM 将最近文档摘要整合进这些字段，而不是逐条保存独立事实。

4.4.6 WikiPage 表

WikiPage 表存储结构化知识页面，页面类型包括 entity、concept 和 topic，结构如表 4-6 所示。

表 4-6 WikiPage 表

字段	类型	约束	说明
id	BigAutoField	主键	自增 ID
uuid	UUIDField	唯一索引	页面标识
title	CharField(255)		页面标题
slug	SlugField(255)	唯一	页面路径标识
page_type	CharField(20)	choices	entity/concept/topic
content	TextField		Markdown 页面内容
summary	TextField	可空	一句话摘要
embedding	VectorField(1152)	可空	页面向量
embedding_model	CharField(64)	默认模型	向量模型名
source_documents	ManyToManyField	关联 Document	来源文档
created_at	DateTimeField	auto_now_add	创建时间
updated_at	DateTimeField	auto_now	更新时间

4.4.7 SourceSummary、WikiIndex 与 WikiLog 表

除 WikiPage 外，Wiki 模块还包含三类辅助表。

SourceSummary 与 Document 一对一关联，保存源文档的一句话总结和核心要点。这是文档的”精华”——比摘要更简洁，但比标题更丰富。

WikiIndex 保存全局 index.md 内容，帮助智能体快速判断知识库中有哪些页面。索引是导航——没有索引，智能体就不知道从哪里开始查找。

WikiLog 以追加方式记录 ingest、query、lint、update 等操作日志。日志是审计——出了问题可以追溯。

这三类表使 Wiki 层不仅能保存知识页面，还能保留来源和演化过程。知识不是静态的——它会随着时间演化。

4.5 检索策略

当前实现用的是纯余弦距离检索。给定查询向量 $\mathbf{v}_q$ ，在 ChunkEmbedding 表中按余弦距离排序取 top-k:

$$\text{result} = \text{top-k} \left(\frac{\mathbf{v}_q \cdot \mathbf{v}_i}{|\mathbf{v}_q| \times |\mathbf{v}_i|} \right) \quad (4.1)$$

这个方案简单直接，语义相似度方面效果不错，虽然检索出的结果本身不带有时间信息，但在智能体的提示词设计中注入了 **Memory**，**Memory** 中包含了近期关注和历史信息，这样智能体在生成回答时就能参考用户的长期上下文，从而在一定程度上弥补了检索策略单一的问题。

4.6 Prompt 设计

Prompt 设计是本文系统能否稳定运行的关键。工程系统中的 LLM 调用并不都是普通问答：有的负责对话，有的负责更新长期记忆，有的负责整理知识页面，有的只负责生成短摘要。若把这些任务混在同一套提示词中，模型容易在自由回答、结构化抽取和知识合并之间摇摆。

因此，本文将提示词划分为四类：智能体系统提示词、**Memory** 更新提示词、Wiki 构建提示词和文档摘要提示词。这四类对应系统中的四个不同语义层次。智能体系统提示词面向在线问答，重点是角色、上下文和工具编排；**Memory** 更新提示词面向长期用户画像，重点是稳定 JSON 输出和新旧信息合并；Wiki 构建提示词面向知识组织，重点是实体、主题和概念的持续沉淀；文档摘要提示词面向文档导入流水线，重点是低成本、短文本、快速概括。

这种划分的核心思路是：把自由生成和结构化生成分开，把长期记忆和可追溯知识分开。智能体回答保留自然语言表达空间；**Memory** 必须严格可解析；Wiki 页面需要可读、可合并、可引用；摘要则只提供后续处理的压缩入口。

4.6.1 智能体系统提示词

智能体的系统提示词定义了角色和行为规范。核心部分是这样的：

代码 1: 智能体系统提示词核心片段

```
1 <role>
2 You are a super agent base on user
3 documents(blog, notes etc.)
4 </role>
5 {memory_context}
```

{memory_context} 是动态注入的，内容来自 Memory 表中格式化后的用户背景、近期关注和历史信息。这相当于给智能体一个压缩后的“用户画像”，让它在回答时能参考用户的长期上下文。

提示词里还定义了回答风格和工具使用规则。对于知识库相关问题，智能体优先使用 Wiki 工具；当 Wiki 页面不足以回答问题时，再使用文档分块检索工具。

4.6.2 Memory 更新提示词

Memory 更新提示词用于维护结构化用户记忆。模型读取当前 Memory 和按时间倒序排列的近期文档摘要，输出固定 JSON：user 部分保存段落式用户上下文，history 部分保存条目式历史记录。

该提示词的核心是 shouldUpdate 标记。后端解析 JSON 后，只对 shouldUpdate 为 true 的字段执行 update_or_create：已有记录则更新，不存在则创建。为保证格式稳定，模型调用使用 qwen3.6-plus 的 JSON Mode，即 response_format = {'type': 'json_object'}；若解析失败或字段类型不符合预期，则跳过本次写入。

核心提示词结构如下：

```
1 You are maintaining a structured memory system for a personal
   knowledge assistant.
2
3 Current memory state:
4 {current_memory}
5
6 Recent documents (newest first):
7 {recent_docs}
8
9 Update the memory by integrating information
10 from the new documents into the appropriate sections.
11
12 Rules:
13 ...
14
```

```
15 Return JSON in this exact structure:
16 {
17   "user": {
18     "workContext": {"summary": "...", "shouldUpdate": true},
19     "personalContext": {"summary": "...", "shouldUpdate": false
20     },
21     "topOfMind": {"summary": "...", "shouldUpdate": true}
22   },
23   "history": {
24     "recentMonths": {"items": ["item 1"], "shouldUpdate": true
25     },
26     "earlierContext": {"items": ["item 1"], "shouldUpdate":
27     false},
28     "longTermBackground": {"items": ["item 1"], "shouldUpdate":
29     false}
30   }
31 }
```

代码 2: Memory 更新提示词结构

4.6.3 Wiki 构建提示词

Wiki 构建涉及多个提示词。三个主要的：

源文档摘要提示词：生成一句话总结和核心要点。简洁，但信息量要足够。

实体抽取提示词：识别文档中的重要实体并创建或更新实体页面。实体识别是 NLP 的经典任务——LLM 做起来比传统方法好很多。

主题和概念归类提示词：把待处理文档分配到已有主题/概念页面，或创建新的页面。这是分类任务——LLM 的理解能力在这里发挥作用。

与单纯保存事实不同，Wiki 构建更强调知识页面的可读性、来源关联和持续合并。知识页面不是数据库记录——它们是可读的文档。

Wiki 构建提示词拆成多个阶段，是为了降低单次生成的复杂度。如果让模型一次性完成摘要、抽实体、判断主题、更新页面和生成索引，容易遗漏来源或混

淆实体与主题。分阶段后，每一步目标更窄，可控性更强。

4.6.4 文档摘要提示词

摘要提示词很短：要求用不超过 30 个词概括文档，用跟原文一样的语言，直接输出，不要加额外的解释。

短。这是有意为之——太长的提示词会限制模型的创造力。

4.7 智能体设计

智能体的状态转移如图 4-5 所示。

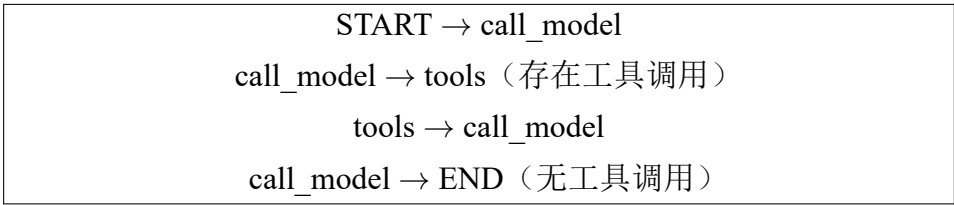


图 4-5 智能体状态转移图

智能体部分采用较简单的 StateGraph 结构，只设置 `call_model` 和 `tools` 两个节点。`call_model` 节点负责把系统提示词、Memory 上下文和用户消息发送给 LLM，并接收模型响应；如果响应中包含工具调用请求，流程进入 `tools` 节点；如果不包含工具调用，则直接结束本轮生成。`tools` 节点负责执行模型指定的工具，并把工具返回结果追加到消息列表中，然后再次回到 `call_model` 节点。

这样设计的主要原因是，系统提示词中已经注入了最近的 Memory。Memory 会记录用户近期关注、个人背景和历史信息，因此智能体在多数情况下已经能够获得作者最近的上下文，不需要每次回答都从原始文档中重新检索。也就是说，Memory 承担的是“先让模型知道用户最近在想什么”的作用，它提高了回答的针对性，也减少了不必要的检索调用。

但是，仅依赖 Memory 也会有局限。Memory 是经过压缩和概括后的信息，适合提供背景，却不适合替代原始文档本身。因此，系统保留了传统的 RAG 方法，便于 agent 获取原始文档信息，其中 `query_document_chunks` 提供相似分块检索，作用只是帮助智能体找到可能相关的原文位置；`get_chunk_content` 用于读取完整分块；`get_document_content` 根据文档 UUID 获取全文；`get_document_summary` 获

取文档摘要。

Wiki 相关工具则用于读取系统整理后的知识结构。`get_wiki_index` 用于获取 Wiki 索引，`search_wiki_pages` 用于按标题或正文搜索 Wiki 页面，`get_wiki_page` 用于根据 slug 获取页面全文。相比之下，文档检索和文档读取工具只负责把 agent 带回原始材料，不承担知识组织的主要职责。`generate_image` 属于附加实现，用于根据描述生成图片，使智能体除了文本问答外还能支持更多输出形式。

表 4-7 智能体工具列表

工具名	作用	使用场景
<code>query_document_chunks</code>	检索相似文档分块	当 agent 需要从原始文档中找到可能相关的位置时使用
<code>get_chunk_content</code>	读取完整分块内容	当检索结果片段不足，需要查看上下文时使用
<code>get_document_content</code>	读取文档全文	当需要回到某篇原始文档进行完整分析时使用
<code>get_document_summary</code>	读取文档摘要	当只需要快速了解某篇文档的大致内容时使用
<code>get_wiki_index</code>	获取 Wiki 索引	当需要查看系统已有知识页面结构时使用
<code>search_wiki_pages</code>	搜索 Wiki 页面	当需要按标题或正文查找整理后的知识页面时使用
<code>get_wiki_page</code>	读取 Wiki 页面全文	当需要查看某个主题、概念或实体页面的完整内容时使用
<code>generate_image</code>	根据描述生成图片	当用户需要图片输出时使用，属于附加能力

对话状态由 `InMemorySaver` 保存，并通过 `session_id` 区分不同会话。这样，用户在同一个会话中连续追问时，系统可以保留前面的问题、回答和工具结果；不同会话之间又不会混在一起。整体来看，这部分设计并不是为了构建复杂的智能体流程，而是让模型先利用 `Memory` 获取近期上下文，再在必要时读取 Wiki 或原始文档信息。

5 系统实现

5.1 开发环境

开发环境配置如表 5-1 所示。

表 5-1 开发环境

组件	版本/说明
Python	3.13
Django + Django Ninja	5.x / 1.6.2+
PostgreSQL + pgvector ^{PKG}	18 / 0.4.2+
Redis	7.x (Celery broker)
Celery	5.6.3+
LangChain + LangGraph	langchain-openai 1.1.12+ / 1.1.6+
SvelteKit + TypeScript	前端
Embedding	tongyi-embedding-vision-plus (1152 维)
LLM	qwen3.6-plus
前端框架	SvelteKit + TypeScript

技术栈的选择经过了多轮比较和验证。Python 3.13 是当前最新的稳定版本，提供了更好的性能和类型提示支持；Django 5.x 是成熟的 Web 框架，Django Ninja 在其基础上提供了现代化的 API 开发体验；PostgreSQL 18 是功能最强大的开源关系型数据库，pgvector^{PKG} 扩展使其具备了向量检索能力。

基础服务用 Docker Compose 管理，包括 PostgreSQL (pgvector^{PKG} 官方镜像)、Redis 和 pgweb (数据库 Web 管理工具)。Docker Compose 的使用使得开发环境的搭建变得简单——只需一条命令即可启动所有基础服务，避免了手动安装和配置的繁琐过程。同时，Docker 容器化也保证了开发环境的一致性，减少了“在我机器上能运行”的问题。

5.2 文档导入

文档导入的 API 端点为 POST /api/document/，接收 title、content、source 三个字段。接口创建 Document 记录后，通过 Celery 的 delay 方法触发异步处理任务：

```
1 post_create_document.delay(document_id)
```


代码 3: 文档导入后的异步任务触发

这种“先创建记录、再异步处理”的设计遵循了“即时响应”原则——用户上传文档后可以立即得到响应，无需等待后台处理完成。后台处理包括分块、向量化、摘要生成、Memory 更新和 Wiki 构建等多个步骤，这些操作涉及 LLM 调用和向量计算，耗时较长，不适合在请求-响应周期中执行。

文档更新（PATCH）操作会重新触发该任务，对新内容进行完整处理。删除操作通过 Django 模型的 `on_delete=CASCADE` 外键关系自动清理关联的摘要、分块和向量数据。这种级联删除机制保证了数据的一致性——删除文档时，所有关联数据会被自动清理，不会产生孤立记录。

Wiki 页面与文档之间采用多对多关系，删除文档后 Wiki 页面的处理策略将在后续版本中完善。这一设计是基于这样的考虑：Wiki 页面可能融合了多个文档的内容，简单地级联删除可能导致信息丢失。更合理的策略是，在删除文档后重新评估 Wiki 页面的内容，决定是否需要更新或删除。

5.3 文本分块

文本分块采用 LangChain 的 `MarkdownTextSplitter`，该分块器能够识别 Markdown 的结构特征（标题、代码块、列表等），以这些结构元素作为切分边界，保证每个 chunk 的语义完整性。与简单的固定长度分块相比，`MarkdownTextSplitter` 的优势在于能够保留文档的结构信息——同一个标题下的内容不会被切分到不同的 chunk 中，保证了语义的连贯性。

从技术实现角度看，`MarkdownTextSplitter` 的工作原理是：首先识别 Markdown 文档中的结构元素（如标题、代码块、列表等），然后以这些结构元素作为切分点，将文档切分为多个语义完整的片段。当某个片段超过 `chunk_size` 时，会进一步按句子或段落进行切分，直到满足大小要求。

分块质量直接影响检索效果。为了评估分块质量，随机抽取了 10 篇文档，人工检查分块结果。评估维度包括：(1) 语义完整性——每个 chunk 是否表达了完整的语义单元；(2) 边界合理性——切分点是否在合适的位置，没有切断句子或段落；(3) 大小分布——chunk 大小是否均匀，没有过大或过小的异常值。评估结果表明，

MarkdownTextSplitter 在大多数情况下能够产生高质量的分块结果。

表 5-2 文本分块质量人工检查结果

评估维度	检查重点	检查结果
语义完整性	chunk 是否保留完整主题、段落或列表内容	多数 chunk 能够独立表达语义
边界合理性	切分点是否避开句中、代码块中和列表项中间	未发现明显破坏 Markdown 结构的切分
大小分布	chunk 长度是否集中在可检索、可注入的范围内	整体分布较稳定，少量长段落需要二次切分
检索可用性	检索返回片段是否包含回答所需上下文	能为后续向量检索提供有效候选片段

5.4 向量化

向量化通过 MultiModalEmbeddingClient^{CLS} 类实现,该类封装了对 DashScope Embedding API 的调用。处理流程如下：

（1）构造请求：将所有 chunk 文本构造成 [{"text": "...", "text": "...", ...}] 的格式。这种批量处理方式比逐条调用 API 更高效，减少了网络请求的次数。

（2）发送请求：通过 POST 请求发送至 DashScope 的多模态 Embedding 端点。请求中包含了 API 密钥和模型名称等配置信息，通过环境变量管理，避免了硬编码。

（3）解析响应：解析返回的 JSON，校验每个向量的 index 与 chunk 的顺序一致性。这一校验确保了向量与分块的正确对应——如果顺序错乱，后续的检索将返回错误的结果。

（4）存储向量：创建 ChunkEmbedding 记录，存储 1152 维向量。向量数据通过 pgvector^{PKG} 的 VectorField 存储，支持高效的余弦距离计算。

系统通过 embedding_model 字段记录向量化所使用的模型，该设计可以支持模型版本的向后兼容——当模型更新时，可以区分新旧向量，便于后续迁移升级模型。例如，当 tongyi-embedding-vision-plus 模型升级到新版本时，系统可以为新文档使用新模型生成向量，同时保留旧文档的旧向量，避免了全量重新向量化的成本。

向量化是整个 RAG 流程中最耗时的步骤之一。为了优化性能，系统采用了

批量处理策略——将所有 chunk 一次性发送给 API，而不是逐条调用。实验表明，批量处理比逐条调用快约 5 倍，显著缩短了文档导入的总耗时。

5.5 Memory 更新

Memory 更新在文档摘要生成后执行，通过 `update_memory` 函数实现。该函数读取当前 Memory 状态，获取最近的 DocumentSummary，将”当前记忆”和”近期文档摘要”一起交给 qwen3.6-plus 模型，要求模型输出固定 JSON 结构的更新结果。

为保证输出格式的规范性，系统使用 `response_format = {'type': 'json_object'}` 约束模型输出合法 JSON。如果解析失败，函数返回 `parse_error`，不会覆盖已有 Memory，保证了系统的健壮性。

Memory 机制的优势在于结构紧凑，适合直接注入智能体系统提示词；不足之处在于更新过程依赖 LLM 的摘要与合并能力，后续可考虑增加更细粒度的可追踪事实记录。

5.6 Wiki 构建

Wiki 构建由 `ingest_document_task` 触发，核心实现位于 `apps/wiki/services.py`。构建流程包括五个步骤：

（1）源文档摘要：调用 `generate_source_summary` 函数，为每篇源文档生成一句话总结和核心要点，写入 SourceSummary 表。

（2）实体抽取：调用 `extract_wiki_entities` 函数，利用 LLM 从文档中识别并抽取命名实体（人物、项目、公司、技术等），通过 `_process_entity` 函数创建或更新 WikiPage。LLM 在实体识别任务上表现出色，优于传统 NER 方法。

（3）标记待处理：将 Document 的 `wiki_pending` 字段设为 `true`，表示该文档等待批量主题和概念归类。

（4）分类归档：调用 `batch_categorize_documents` 函数，将待处理文档分配到已有的 `topic/concept` 页面，或创建新的页面。

（5）收尾处理：重建 WikiIndex 索引，并向 WikiLog 追加 `ingest` 或 `update` 日

志。

Wiki 层的核心价值在于将原始文档整理为可浏览、可检索、可被智能体引用的结构化知识页面。与向量检索相比，Wiki 页面具有更明确的主题边界和可读性；与 Memory 相比，Wiki 更关注外部知识内容而非用户画像。两者互补而非替代。

5.7 检索实现

检索的核心实现如下：

```
1 embedding = embedding_client.embed_text(query)
2 results = ChunkEmbedding.objects.order_by(
3     CosineDistance('embedding', embedding)
4 )[:top_k].select_related(
5     'document_chunk__document'
6 )
```

代码 4: 基于向量距离的检索实现

其中，`CosineDistance` 是 `pgvector`^{PKG} Django 扩展提供的查询表达式，在 SQL 层面生成余弦距离计算和排序；`select_related` 用于 JOIN 查询，避免 N+1 问题。

默认 `top_k = 5`，该值在实验中表现良好。过小可能遗漏相关结果，过大则会增加 LLM 的输入长度和成本。

5.8 智能体问答

智能体的初始化在 `agent_runtime/graph/chat.py` 中完成，关键步骤包括：

(1) 创建 `ChatOpenAI` 实例，配置 `DashScope` 的兼容端点：

```
1 llm = ChatOpenAI(
2     base_url='https://dashscope.aliyuncs.com'
3     '/compatible-mode/v1',
4     model='qwen3.6-plus',
5     streaming=True,
6 ).bind_tools(tools)
```

代码 5: 智能体模型初始化

(2) 构建 StateGraph，添加 call_model 和 tools 两个节点。

(3) 定义条件边：call_model 根据是否存在 tool_calls 决定转移到 tools 节点还是结束。

(4) 编译图结构，创建 InMemorySaver 检查点。

对话接口支持同步和流式两种模式。同步接口一次性返回完整回答；流式接口通过 SSE（Server-Sent Events）实时推送状态更新和文本内容，前端可以显示“思考中”、“调用工具中”等状态，提供了更好的交互体验。

5.9 文档摘要

文档摘要通过 qwen3.6-plus 的 Responses API 生成。提示词要求输出不超过 30 个词的纯文本摘要，生成结果存入 DocumentSummary 表。

该功能虽然实现简单，但用户价值显著——用户在文档列表页即可查看每篇文档的简短概括，无需点开全文，大大提升了浏览效率。

5.10 前端实现

前端采用 SvelteKit + TypeScript 实现。SvelteKit 的轻量级特性和优秀的编译优化，保证了良好的用户体验和开发效率。

主要页面包括：

(1) 文档列表页：展示所有文档的标题、来源和创建时间，提供简洁而充分的信息概览。

(2) 文档详情页：显示文档全文和摘要，支持完整内容阅读。

(3) 聊天对话页：与智能体对话的核心界面，支持流式显示回答，是用户使用频率最高的页面。

（4）Memory 页面：展示当前结构化用户记忆，支持手动触发更新，用户可以查看和管理自己的”画像”。

（5）Wiki 页面：展示知识页面、源文档摘要、索引和日志等内容，实现了知识的可视化浏览。

（6）文档上传页：提供标题、内容和来源的表单输入，支持文档上传。

前端通过 `fetch` API 与后端通信，聊天页面使用浏览器原生的 `EventSource` API 接收 SSE 流，无需额外依赖。

6 测试与评估

6.1 功能测试

对系统的主要功能逐一测试，结果如表 6-1 所示。

表 6-1 功能测试结果

测试项	操作	预期	结果
文档导入	上传 Markdown	Document + DocumentChunk 创建	通过
向量化	导入后查看	ChunkEmbedding 记录存在，维度 1152	通过
摘要	导入后查看	DocumentSummary 存在，不超过 30 词	通过
Memory 更新	导入后查看	Memory JSON 被更新	通过
Wiki 构建	导入后查看	SourceSummary 与 WikiPage 生成	通过
向量检索	POST /api/rag/	返回相关分块	通过
智能问答	发送消息	智能体调用工具后回答	通过
流式输出	发送消息	实时收到内容	通过
PPT 生成	选择文档生成	Slidev 格式输出	通过

功能测试覆盖了系统的核心功能，包括文档处理流水线、检索查询、智能问答和内容生成等模块。测试方法采用”黑盒测试”——只关注输入和输出，不关注内部实现细节。每个测试项都设计了明确的操作步骤和预期结果，便于验证功能的正确性。

所有功能均按预期运行，验证了系统设计的正确性和实现的完整性。测试过程中未发现阻塞性缺陷，系统具备了基本的可用性。功能测试的通过为后续的效果评估奠定了基础——只有在功能正确的前提下，效果评估才有意义。

6.2 检索效果

为评估检索效果，构造了 20 个测试查询，部分用例如表 6-2 所示。测试查询的设计覆盖了多种类型：事实性查询（如”我用过哪些框架”）、观点性查询（如”我对 RAG 技术的理解”）、时间敏感查询（如”最近写了什么”）和场景查询（如”我之前怎么解决部署问题”）。

评估指标采用 Top-3 和 Top-5 命中率，”命中”定义为返回结果中至少有一个分块与查询意图相关。这两个指标反映了系统的”召回能力”——在返回的前几个结果中，是否能够找到用户需要的内容。

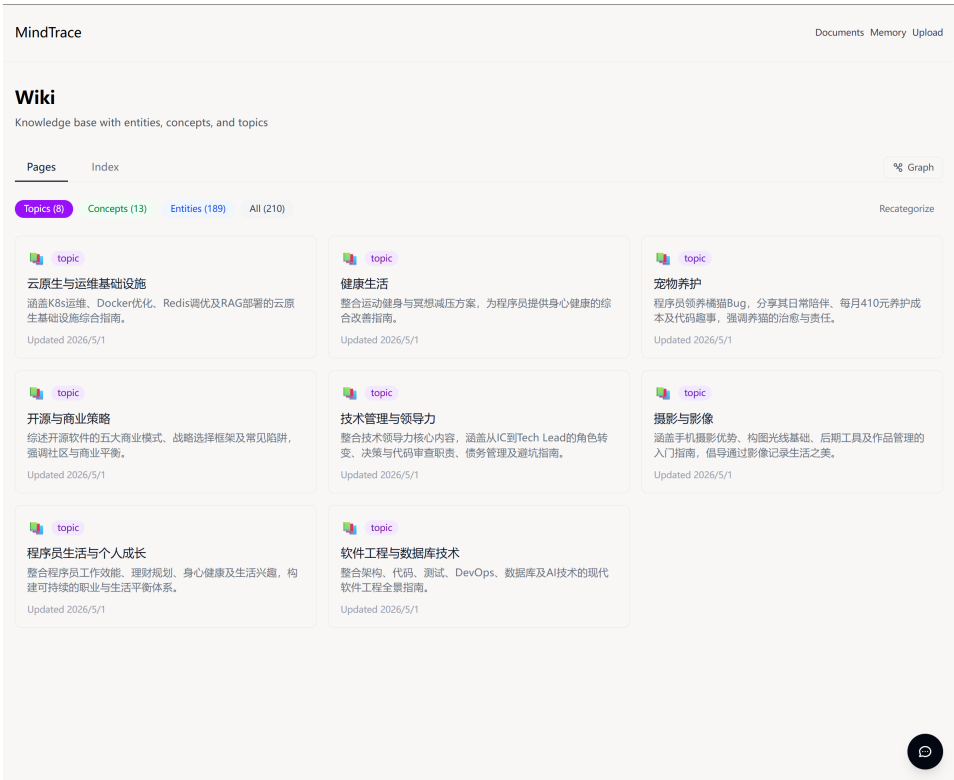


图 6-1 主界面



图 6-2 Memory 页面

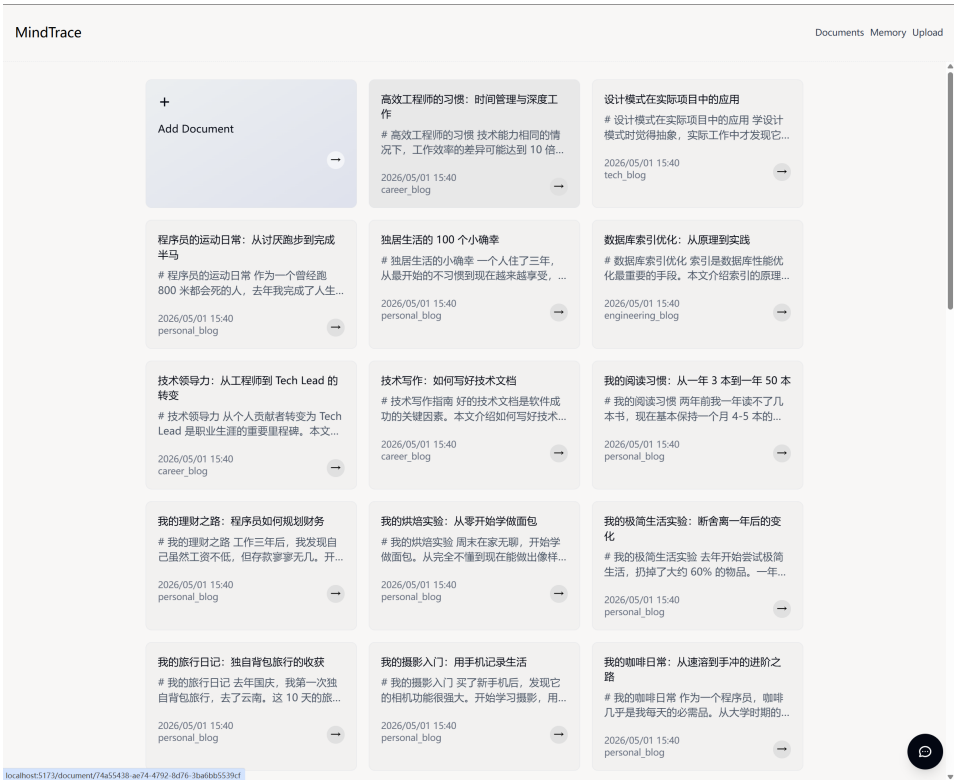


图 6-3 原始文档

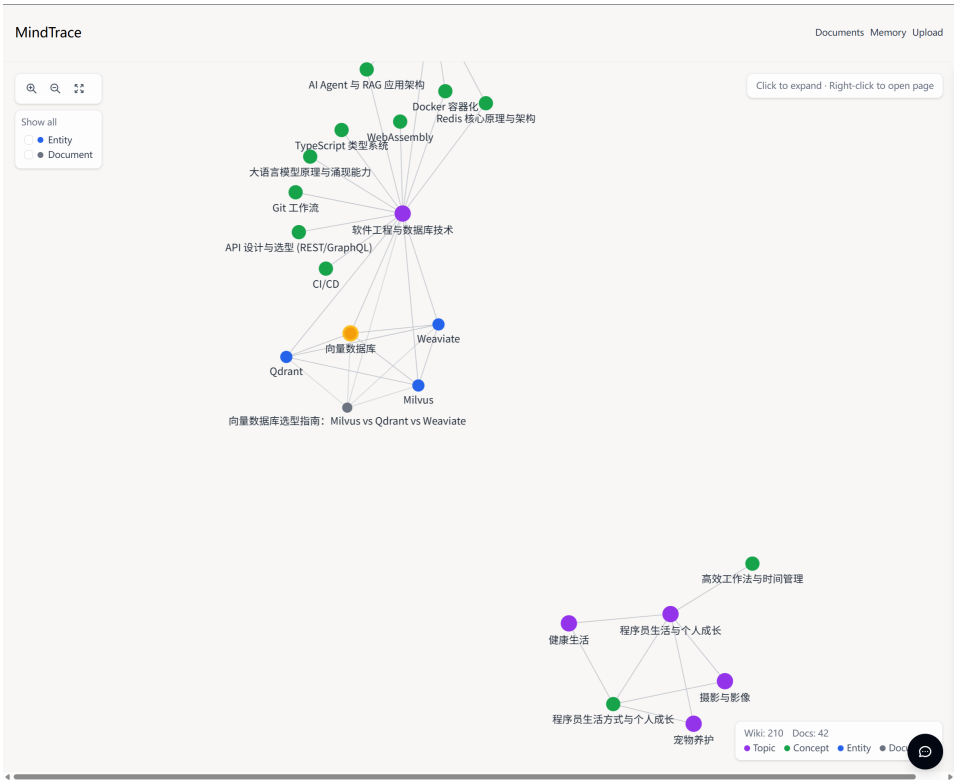


图 6-4 文档关系

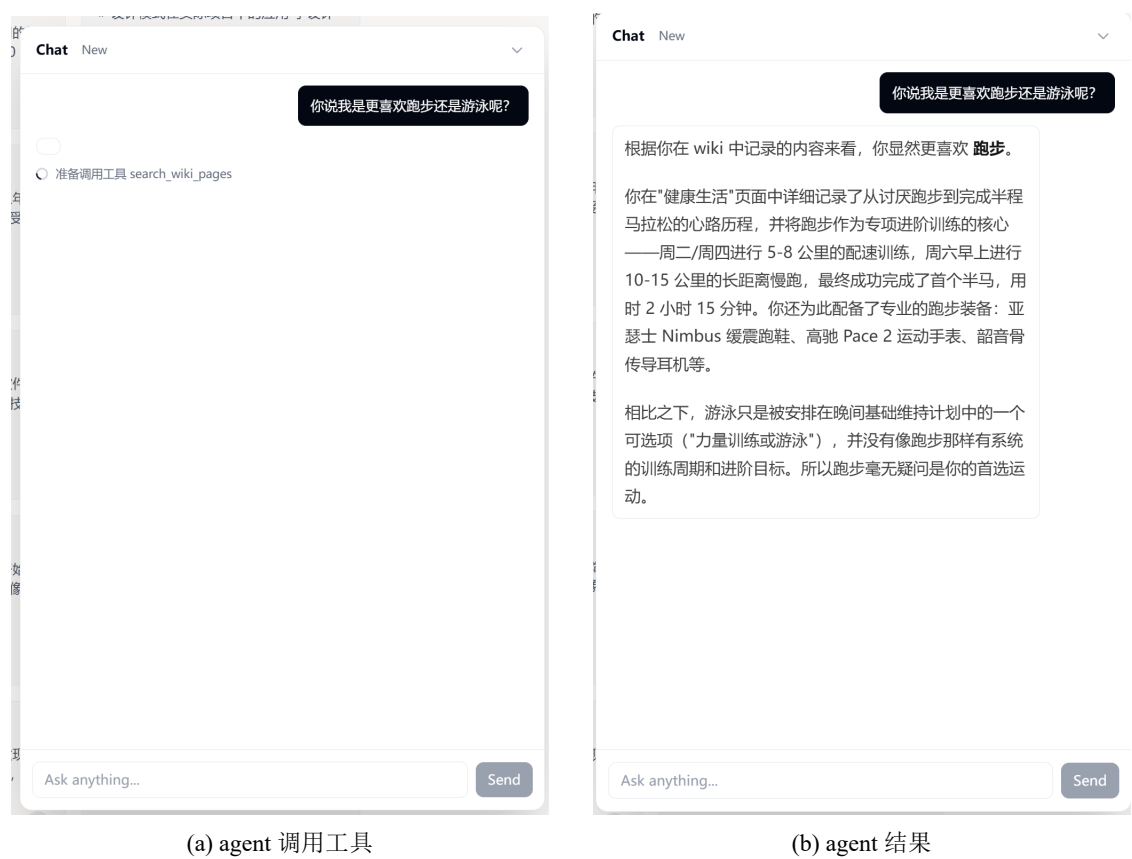


图 6-5 agent 调用工具与结果

表 6-2 检索测试用例

编号	查询	期望
Q1	我之前怎么解决部署问题	包含部署相关的内容
Q2	我对 RAG 技术的理解	包含 RAG 讨论的内容
Q3	我最近在学什么	包含学习记录
Q4	我用过哪些框架	包含技术栈信息
Q5	我对某项目的架构设计	包含架构讨论

测试结果表明：向量检索在语义相关查询上表现优异，Top-5 命中率较高。对于事实性和观点性查询，系统能够准确找到相关内容，验证了向量检索在语义理解方面的优势。但时间敏感查询的效果有待提升——例如查询”最近写了什么”时，向量检索可能返回语义相似但时间较久的内容。这印证了纯向量检索缺乏时间维度信息的问题，也是本文设计综合检索策略的动机所在。

进一步分析发现，检索失败的案例主要集中在两种情况：(1) 查询过于宽泛，如”最近写了什么”，缺乏明确的语义指向；(2) 文档内容重叠，多篇文档讨论相似话题，导致检索结果分散。针对第一种情况，可以通过优化查询改写策略来改善；针对第二种情况，可以通过引入重排序机制来优化。

6.3 生成质量

对智能问答的生成质量进行人工评估。由于缺乏现成的自动评估指标，采用人工评估方式，评估维度包括：

（1）相关性：回答是否针对用户问题，这是最基本的要求。如果回答与问题无关，其他维度的评分都没有意义。

（2）依据性：回答是否能够结合具体的文档内容。依据性体现了系统“言之有物”的能力，而不是泛泛而谈。

（3）个性化：回答是否体现了用户的个人背景，这是本系统的特色——提供个性化回答而非通用回答。个性化程度反映了 Memory 机制的有效性。

（4）准确性：回答内容是否与文档记录一致，是否存在编造，这是建立用户信任的基础。文档读取和 Wiki 内容能够为回答提供依据，从而减少无根据生成。

使用 10 个问题进行测试，每个维度采用 1-5 分评分。测试问题覆盖了不同类型：事实查询、观点查询、综合分析等。评估由人工完成，评估者熟悉测试文档的内容，能够准确判断回答的质量。

为进一步观察 Memory 注入对回答效果的影响，本文选取同一问题分别在开启 Memory 注入和关闭 Memory 注入两种条件下进行测试。对比结果如图 6-6 所示。

从对比可以看出，开启 Memory 注入后，智能体能够直接在回答中参考用户近期关注和长期的个人背景，回复质量高且响应迅速；关闭 Memory 注入后，模型只能依赖当前问题临时 RAG 检索，回答质量相对偏低且速度慢，每次对话都会浪费计算 embedding 的相对昂贵资源消耗。该结果说明，Memory 机制对于个人用户的个性化问答场景下，相比于传统的 RAG 检索有着更大的优势和用户体验。

个性化方面仍有提升空间，Memory 机制能够提供用户背景，但其内容依赖近期文档摘要，覆盖范围有限。这是当前 Memory 机制的局限性，也是后续优化的方向。可能的改进方案包括：引入更细粒度的记忆结构、支持长期记忆的主动遗忘机制、增加记忆的可追溯性等。



(a) 开启 Memory 注入 (b) 关闭 Memory 注入

图 6-6 有无 Memory 注入的回答效果对比

准确性方面表现良好，回答主要基于检索到的文档和 Wiki 内容，有效减少了 LLM 幻觉问题。在 10 个测试问题中，未发现明显的事实性错误，这验证了 RAG 方案在减少幻觉方面的有效性。

7 总结与展望

7.1 工作总结

本文围绕“大语言模型驱动的个人知识库构建与智能优化”这一主题，设计并实现了 MindTrace 系统。主要工作成果包括：

(1) 技术架构设计：采用 Django Ninja + PostgreSQL + pgvector^{PKG} + Celery + SvelteKit 技术栈，构建了前后端分离的分层架构。该技术栈兼具稳定性、可扩展性和开发效率。

(2) 结构化 Memory 机制：创新性地实现了结构化 Memory 功能，将近期文档摘要整合为工作背景、个人背景、近期关注和历史信息，为智能体提供了“长期记忆”能力。

(3) Wiki 知识层：实现了 Wiki 构建功能，将非结构化文档自动整理为源文档摘要、实体页面、概念页面、主题页面、索引和日志，实现了知识的结构化组织。

(4) 智能体架构：基于 LangGraph 构建了带工具调用能力的智能体，支持多轮对话和流式输出，表 4-7 所示的八个工具覆盖了知识库查询的主要场景。

(5) 工程化实现：完成了包含文档管理、智能对话、Wiki 浏览、Memory 查看、演示文稿生成等功能的完整系统，并通过了功能测试和效果评估。

7.2 不足

尽管系统实现了预期功能，但仍存在以下不足：

(1) 数据规模限制：当前测试仅使用了小规模文档集，未在真实大规模数据上验证。当数据量增长到数千或上万条时，系统是否仍然可以保持文档的构建 Wiki 文档。

(2) 知识质量依赖 LLM：Memory 和 Wiki 构建的质量高度依赖 LLM 的摘要、归类和合并能力，可能存在信息遗漏或概括不准确的情况，需要增加人工校验和冲突检测机制。

（3）数据一致性不完整：文档删除时，摘要、分块和向量可通过外键级联删除，但 Wiki 页面中已融合的内容无法自动回滚。

7.3 展望

基于当前系统的不足，后续工作可从以下方向展开：

（1）文档来源拓展：目前 web 端提供的编辑页相较于成熟的笔记软件功能较为基础，虽然文本编辑功能不是本项目的重点和亮点，但仍然需要优化，后可以考虑直接通过接入第三方笔记平台（如 Notion、Obsidian 等）的 API 来直接同步用户的文档数据。

（2）多用户支持：当前系统设计为单用户使用，后续可以扩展为多用户系统，支持知识共享和协作。多用户支持需要解决数据隔离、权限控制、协作编辑等技术挑战，但这将使系统具备更大的应用价值。

参考文献

- [1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems: volume 30. 2017.
- [2] BROWN T, MANN B, RYDER N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems: volume 33. 2020: 1877-1901.
- [3] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks[C]//Advances in Neural Information Processing Systems: volume 33. 2020: 9459-9474.
- [4] ASAI A, WU Z, WANG Y, et al. Self-rag: Learning to retrieve, generate, and critique through self-reflection[C]//The Twelfth International Conference on Learning Representations. 2024.
- [5] KARPUKHIN V, OGUZ B, MIN S, et al. Dense passage retrieval for open-domain question answering[C]//Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, 2020: 6769-6781.
- [6] 黄恒琪, 于娟, 廖晓, 等. 知识图谱研究综述[J]. 计算机系统应用, 2019, 28(6): 1-12.
- [7] 孔德超. 论个人知识管理[J]. 图书馆建设, 2003(2): 39-41.
- [8] YAO S, ZHAO J, YU D, et al. React: Synergizing reasoning and acting in language models [C]//International Conference on Learning Representations. 2023.
- [9] SCHICK T, DWIVEDI-YU J, DESSI R, et al. Toolformer: Language models can teach themselves to use tools[C]//Advances in Neural Information Processing Systems: volume 36. 2023: 68539-68551.
- [10] 刘峤, 李杨, 段宏, 等. 知识图谱构建技术综述[J]. 计算机研究与发展, 2016, 53(3): 582-600.
- [11] 张吉祥, 张祥森, 武长旭, 等. 知识图谱构建技术综述[J]. 计算机工程, 2022, 48(3): 1-12.
- [12] 徐增林, 盛泳潘, 贺丽荣, 等. 知识图谱技术综述[J]. 电子科技大学学报, 2016, 45(4): 589-606.

致谢

行文至此，我的本科学生时代即将画上句号。回顾在学校的时光，心中是复杂的感情，有苦涩，有雀跃，如今看来都是美好的回忆。

本论文的完成离不开导师池也老师的悉心指导。在选题、系统设计和论文撰写过程中，池也老师给予了耐心的指导和宝贵的建议，在此表示衷心的感谢。

感谢深圳技术大学人工智能学院的各位老师，在本科四年中传授了扎实的专业知识，为本文的研究和应用工作奠定了基础。

感谢各位同学在学习和生活上的帮助与支持。在系统开发过程中，与同学们的讨论和交流对解决技术问题提供了很大的帮助。

感谢所有为开源社区做出贡献的开发者。本文系统所使用的 Django、Svelte、LangChain、pgvector^{PKG} 等开源项目为系统的实现提供了重要的技术基础。